Lesson 6

3D GRAPHICS

3D Modeling Transformations

- Methods for object modeling transformation in three dimensions are extended from two dimensional methods by including consideration for the z coordinate.
- Straightforward for translation and scale, rotation more difficult
- Homogeneous coordinates: 4 components

- Transformation matrices: 4×4 elements

3D Point Homogenous Coordinate

- A 3D point **P** is represented in homogeneous coordinates by a 4-dimensional Vector
- The main advantage: it is easier to compose translation and rotation
- Everything is matrix multiplication

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{P} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D Transformation

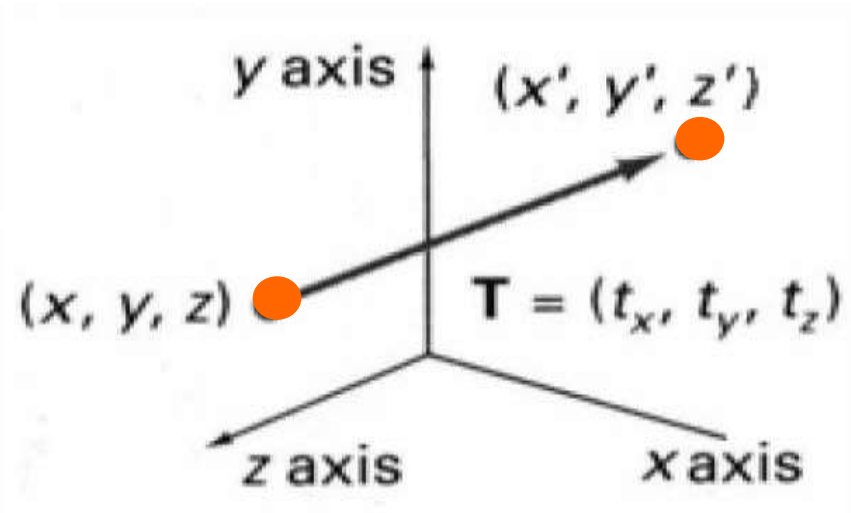
- In homogeneous coordinates, 3D transformations are represented by 4×4 matrixes:

$$\begin{bmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D Translation

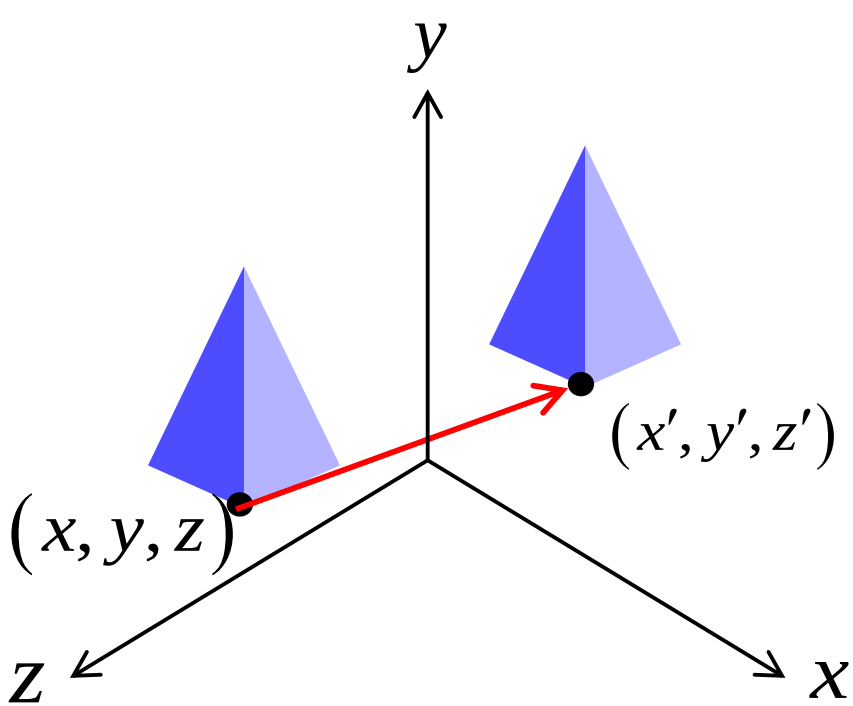
- **P** is **translated** to **P'** by:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



3D Translation

Very similar to 2D. Using 4x4 matrices rather than 3x3.



$$x' = x + t_x$$

$$y' = y + t_y$$

$$z' = z + t_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D Rotation

- Rotations are specified by a **rotation axis** and an **angle**
- The easiest **rotation axes** are those that are parallel to the coordinate axis.
- Positive rotation **angles** produce counterclockwise rotations about a coordinate axis, if we are looking along the positive half of the axis toward the coordinate origin.

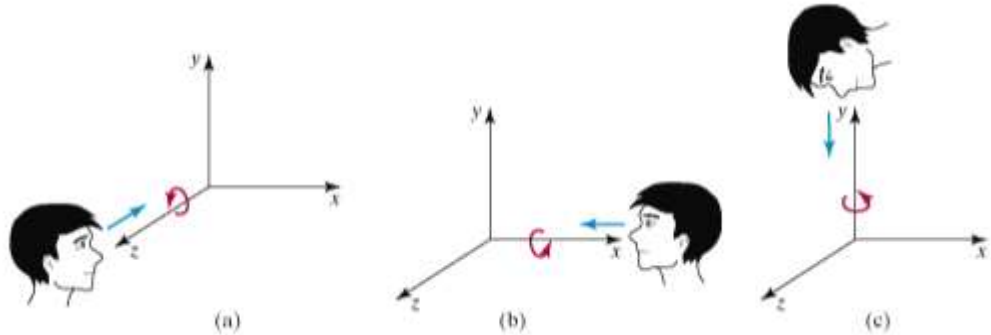


Figure 5-36

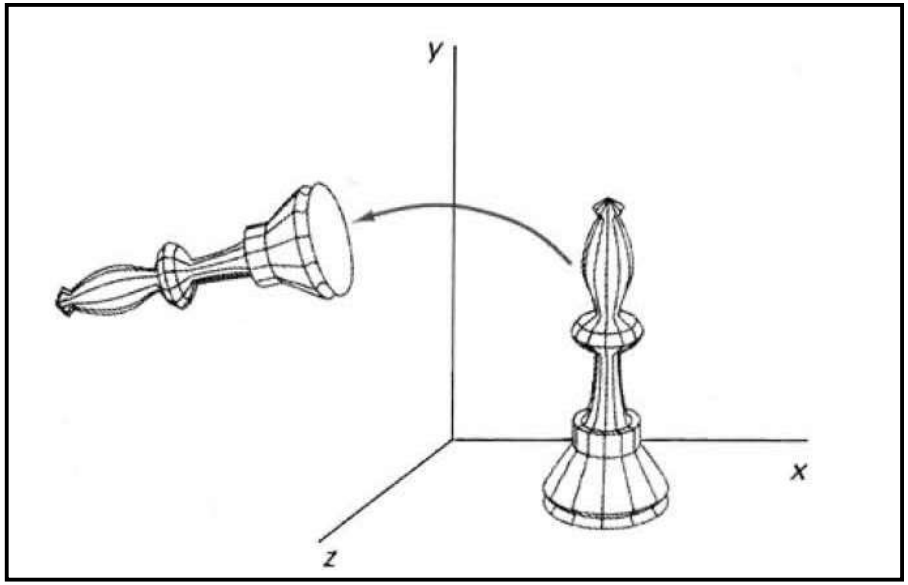
Positive rotations about a coordinate axis are counterclockwise, when looking along the positive half of the axis toward the origin.

Coordinate Axis Rotations

- **Z-axis rotation:** For z axis same as 2D rotation:

$$\mathbf{P}' = \mathbf{R}_z(\theta) \cdot \mathbf{P}$$

$$\boxed{\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}}$$

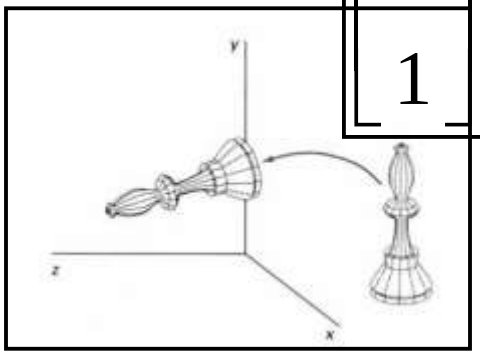


Coordinate Axis Rotations

- **X-axis rotation:**

$$\mathbf{P}' = \mathbf{R}_x(\theta) \cdot \mathbf{P}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

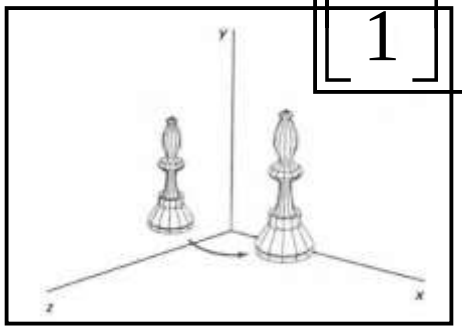


Coordinate Axis Rotations

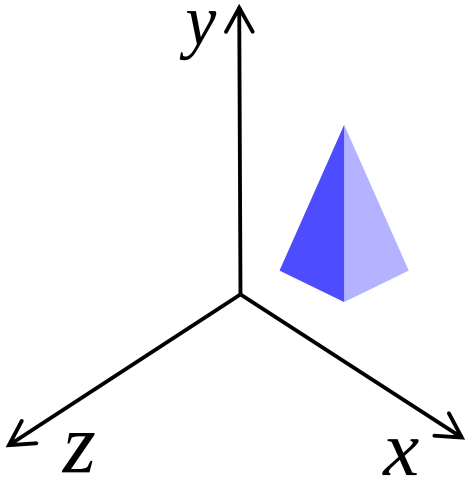
- **Y-axis rotation:**

$$\mathbf{P}' = \mathbf{R}_y(\theta) \cdot \mathbf{P}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



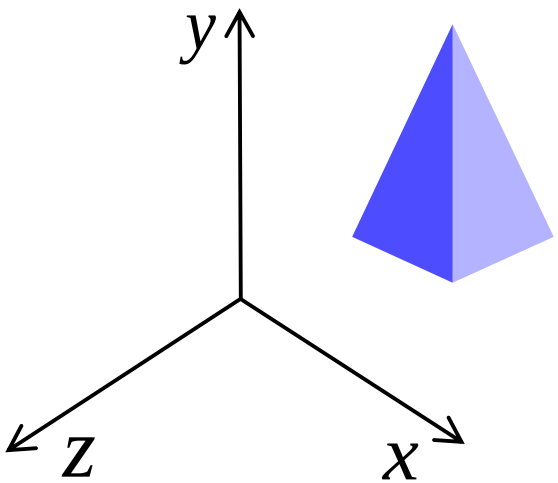
3D Scaling



$$x' = x \cdot S_x$$

$$y' = y \cdot S_y$$

$$z' = x \cdot S_z$$



Changes the size of the object and repositions the object relative to the coordinate origin.

$$\mathbf{P}' = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \mathbf{S} \cdot \mathbf{P}$$

3D Rendering Pipeline

3D Geometric Primitives

↓

Modeling Transformation — Transform into 3D world coordinate system

↓

Lighting — Transform into 3D camera coordinate system
Done with modeling transformation

↓

Viewing Transformation — Illuminate according to lighting and reflectance
Apply texture maps

↓

Projection Transformation — Transform into 2D screen coordinate system

↓

Clipping — Clip primitives outside camera's view

↓

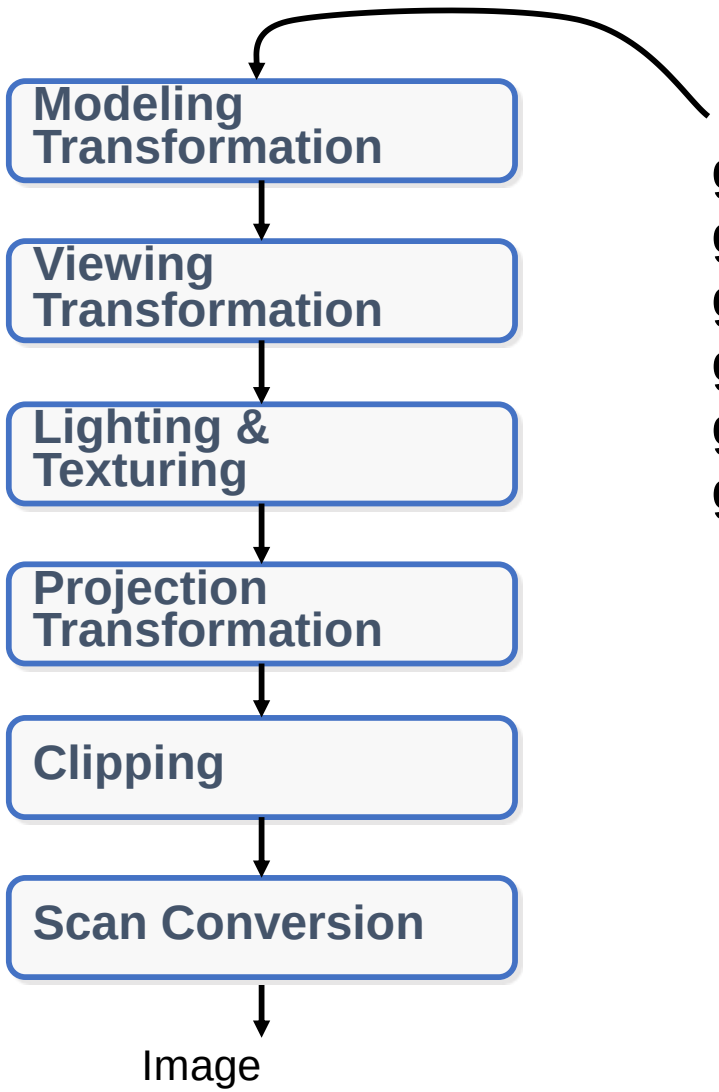
Scan Conversion — Draw pixels (includes texturing, hidden surface, ...)

↓

Image

(this pipeline applies only for direct illumination)

Example: OpenGL

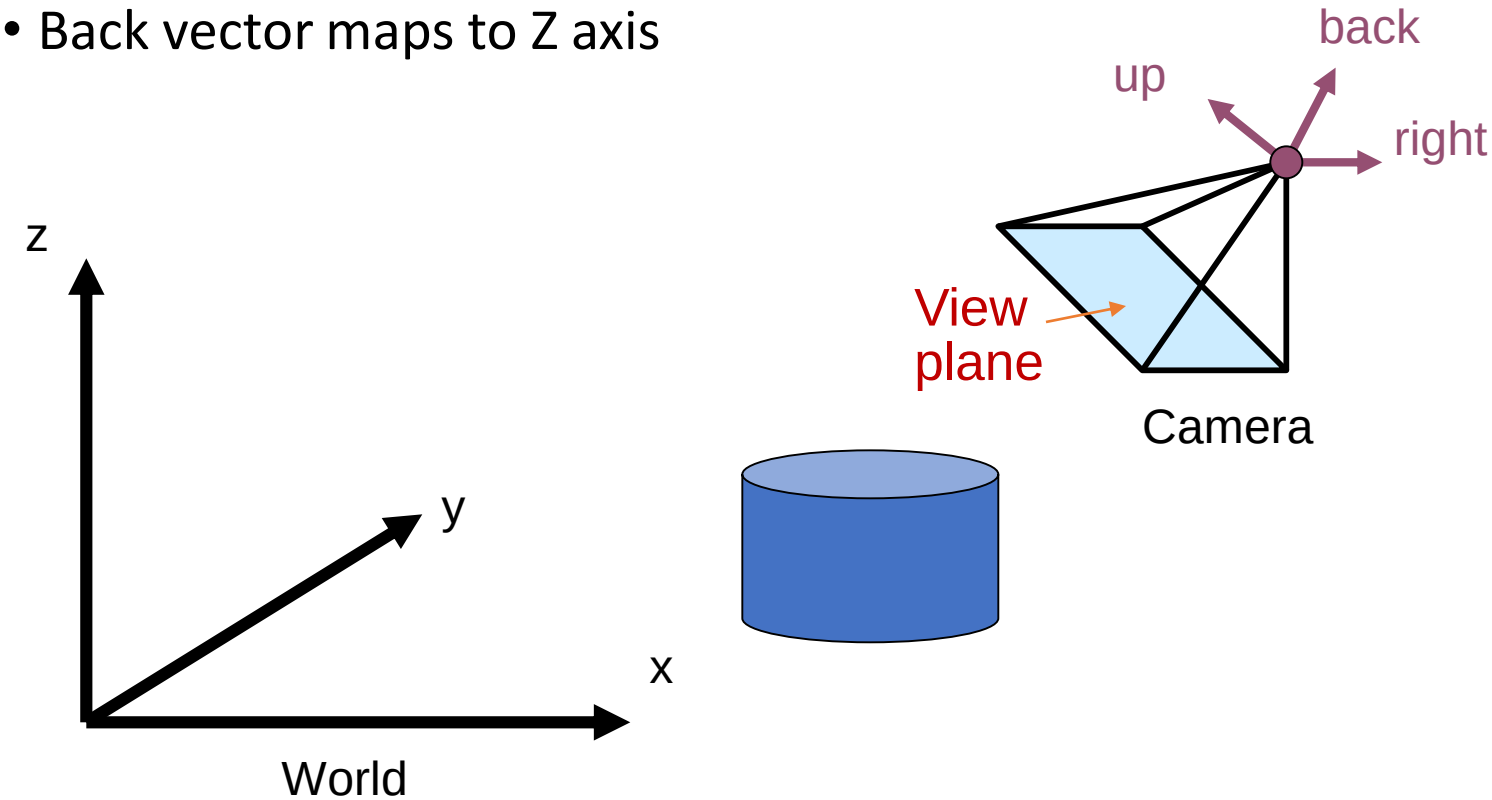



```
glBegin(GL_POLYGON);
glVertex3f(0.0, 0.0, 0.0);
glVertex3f(1.0, 0.0, 0.0);
glVertex3f(1.0, 1.0, 1.0);
glVertex3f(0.0, 1.0, 1.0);
glEnd();
```

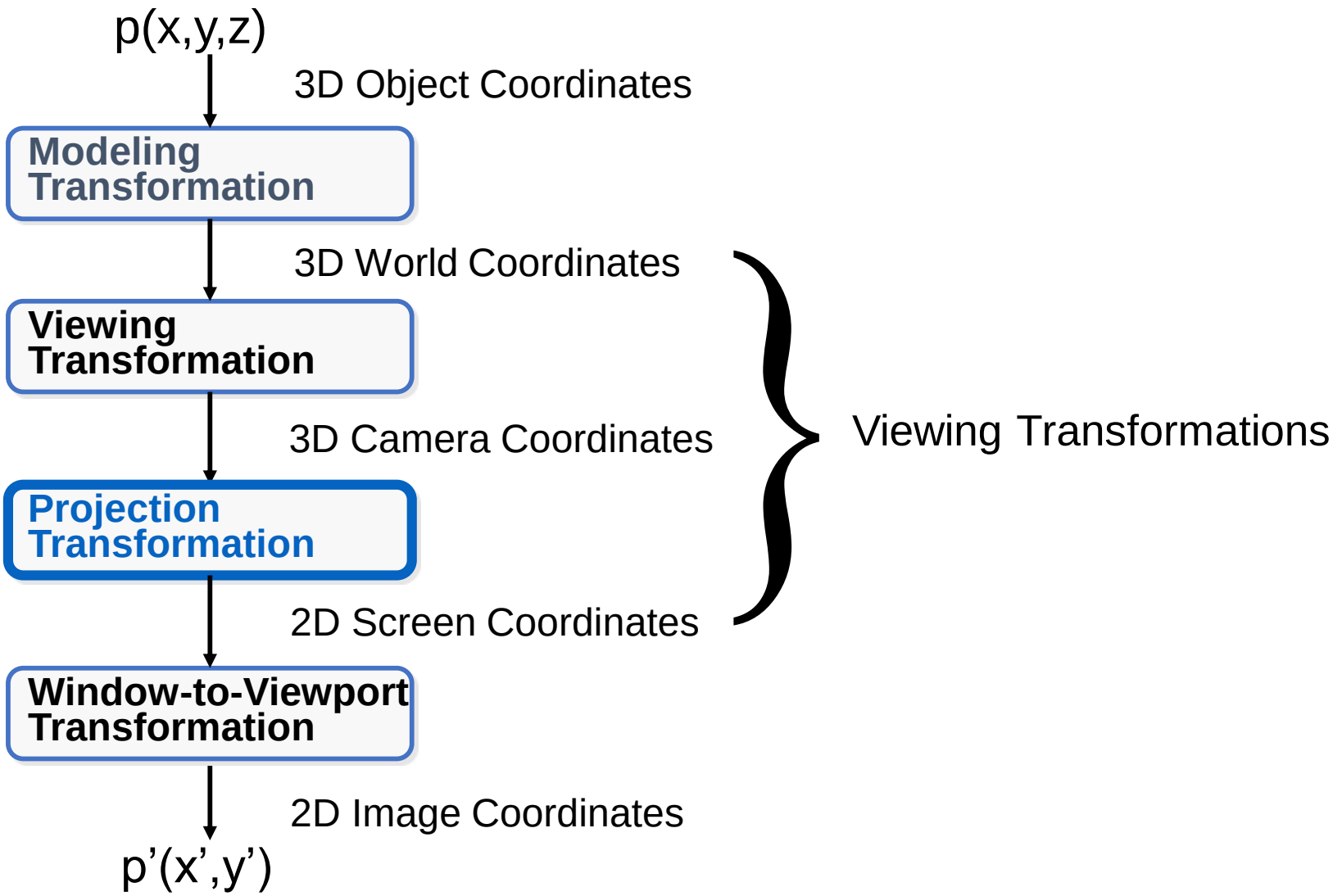
OpenGL executes steps of 3D rendering pipeline for each polygon

Viewing Transformation

- Mapping from world to camera coordinates
  - Eye position maps to origin
  - Right vector maps to X axis
  - Up vector maps to Y axis
  - Back vector maps to Z axis



Viewing Transformations



Projection

- General definition: Transform points in *n*-space to *m*-space ($m<n$)

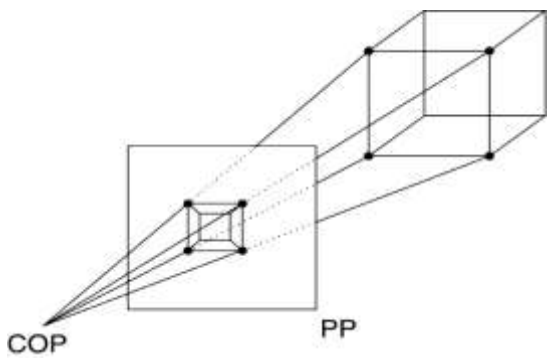
- In computer graphics: Map 3D camera coordinates to 2D screen coordinates

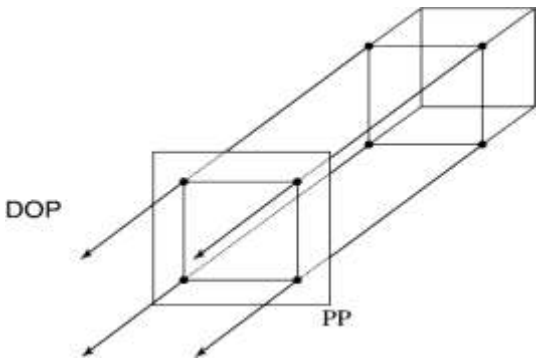
- In 3D, we map points from 3-space to the **projection plane** (PP) (a.k.a., image plane) along **projectors** (a.k.a., viewing rays) emanating from the center of projection (COP)

- There are two basic types of projections:
- Perspective – distance from COP to PP finite
- Parallel – distance from COP to PP infinite

Parallel vs Perspective Projection

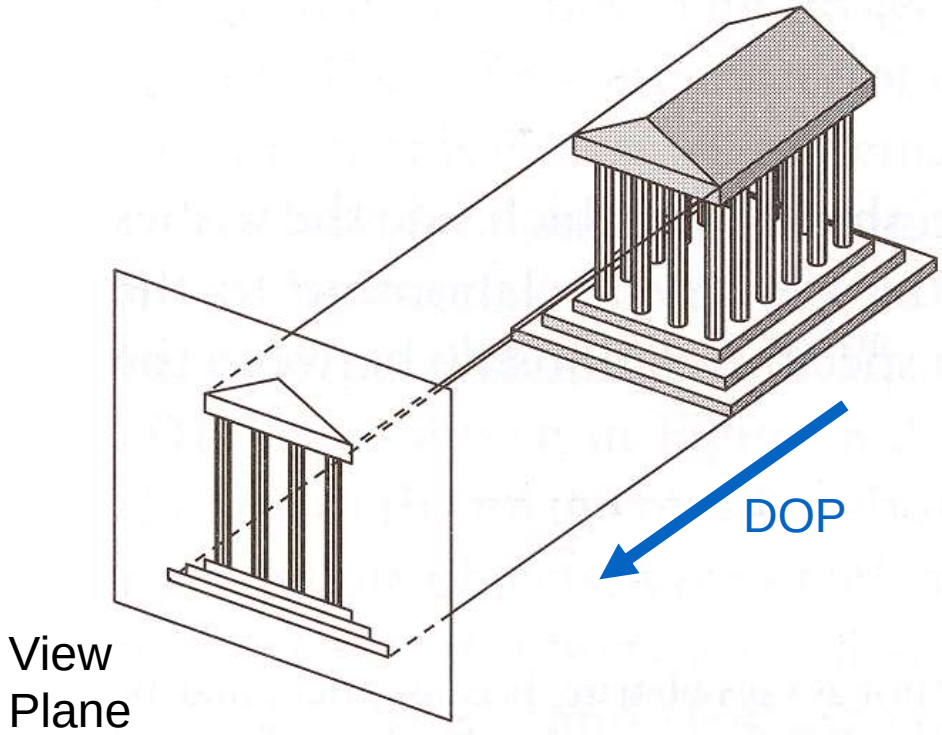
- Parallel Projection
  - Preserve size
    - Good for determining relative size
- Perspective Projection
  - Projection along rays, no two “rays” are parallel to each other
  - Closer objects appears larger
  - Human vision!





Parallel Projection

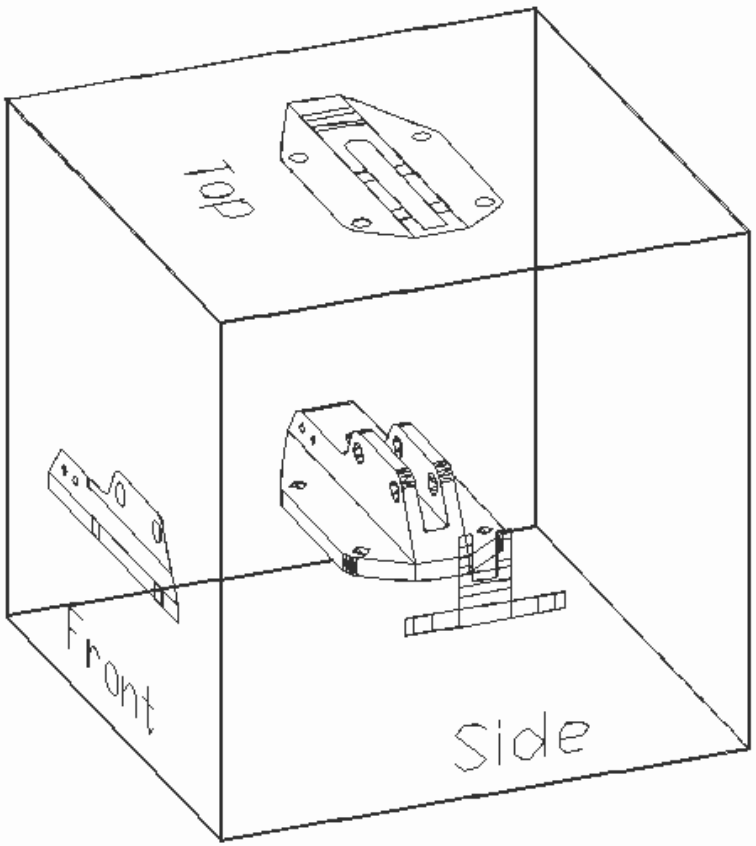
- For parallel projections, we specify a **direction of projection** (DOP) instead of a COP
- Center of projection is at infinity
  - Direction of projection (DOP) is the same for all points



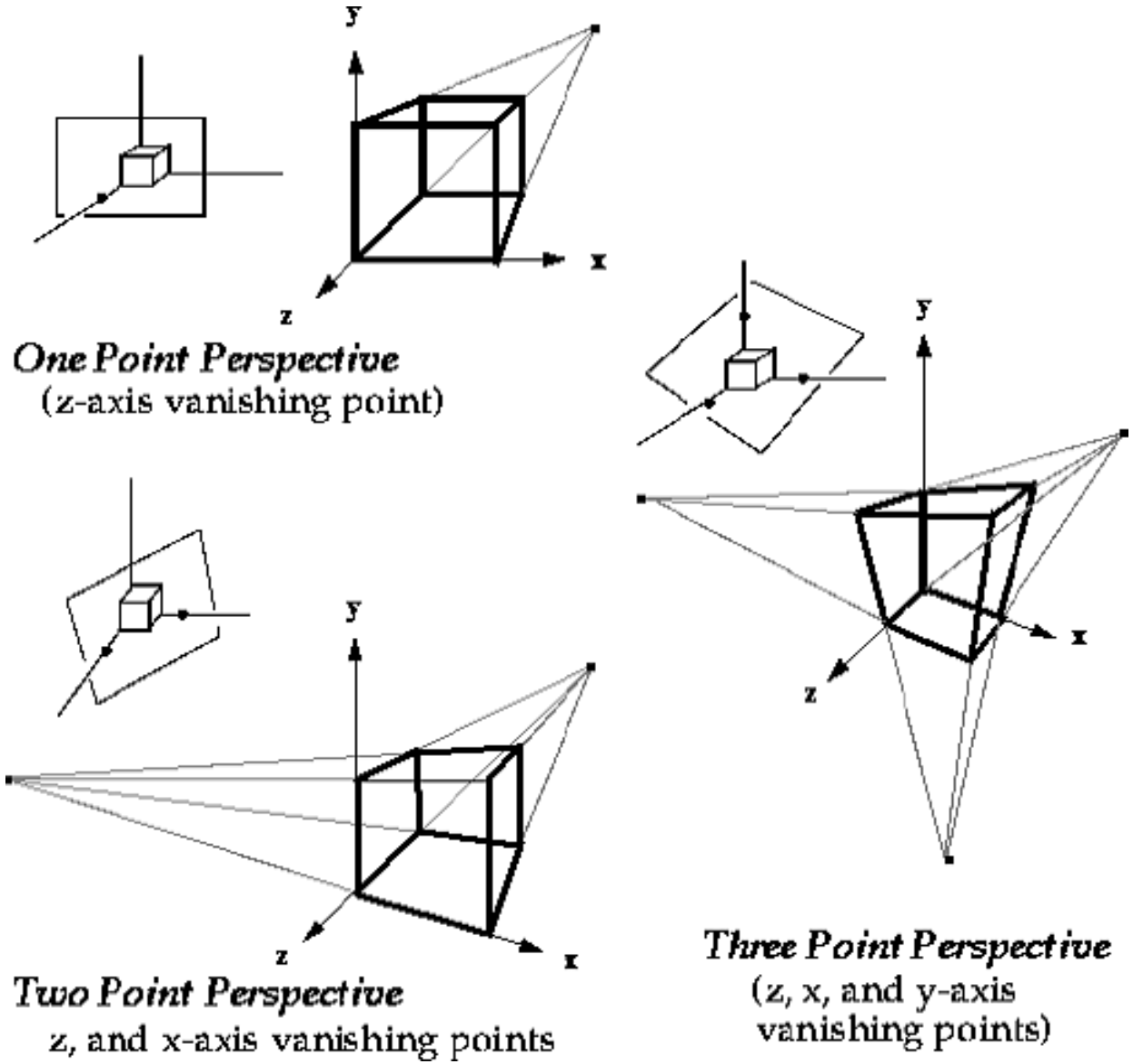
Properties of parallel projection

- Properties of parallel projection:
  - Are actually a kind of affine transformation
    - Parallel lines remain parallel
    - Ratios are preserved
    - Angles not (in general) preserved
  - Not realistic looking
  - Good for exact measurements,
    Most often used in
    - CAD,
    - architectural drawings,
    - etc.,

    where taking exact measurement
    is important



Perspective projections

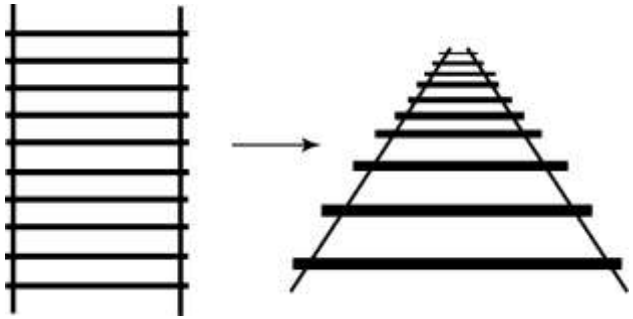


Properties of perspective projections

- Properties of projective transformations:
  - Lines map to lines
  - Parallel lines do <u>not</u> necessarily remain parallel
  - Ratios are <u>not</u> preserved

- One of the advantages of perspective projection is that size varies inversely with distance – looks realistic.

- A disadvantage is that we can't judge distances as exactly as we can with parallel projections.



Perspective Parameters

- There are three different ways to describe a perspective camera
  - Clipping planes, Field of View, Focal distance,
- The most general is clipping planes – they directly describe the region of space you are viewing
- For most graphics applications, field of view is the most convenient
- You can convert one thing to another